From Ontologies and Representations to Evidence

A multi-technology validation architecture for ontology engineering

Trust the evidence, not the file.

WHAT WE ARE VALIDATING, AND WHY THAT IS A THEORETICAL ISSUE

Luís Antonio de Almeida Rodriguez - PhD - Aeronautics Institute of Technology (ITA)

Goal

Goal = evidence architecture.

To present to the audience the theory and practical architecture of a validation process that transforms generated OWL/RDF representations into traceable, executable, and reproducible evidence packages.

- Explain why parsing RDF/TTL is not enough
- Show a way to mix OWL, SPARQL, SHACL, HermiT, ABoxes, and reports
- Describe the advantage of every phase and deliverable in this process
- Give audience a validation model they can reuse.

Summary

Five movements.

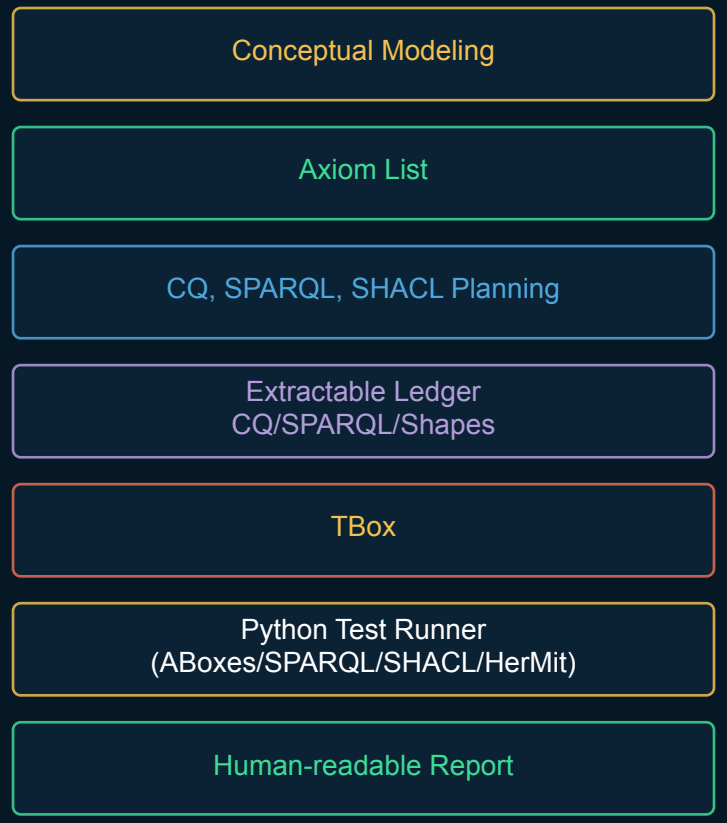
- The problem: RDF valid file is not validated knowledge
- The artifact chain
  - Conceptual Modeling → Axiom List → Planning → Ledger→ TBox→ Python runner
- The execution chain: RDF/TTL → ABoxes → Python parsing → HermiT → SPARQL → SHACL shapes
- The evidence chain: Made in Python - human-readable reports, ABoxes profiles, expected and formal results
- The advantage: reproducibility, explainability, and controlled semantic evolution.

The Core Thesis

Layered claims, many evidences.

Ontology validation must be layered because ontology claims are layered.

- A syntax claim needs a parser
- A logical claim needs OWL reasoning
- A competency question claim needs SPARQL
- A data-conformance claim needs SHACL shapes
- A scientific claim needs traceable explanation and reproducibility



The Starting Problems

Loading and parsing is not knowing.

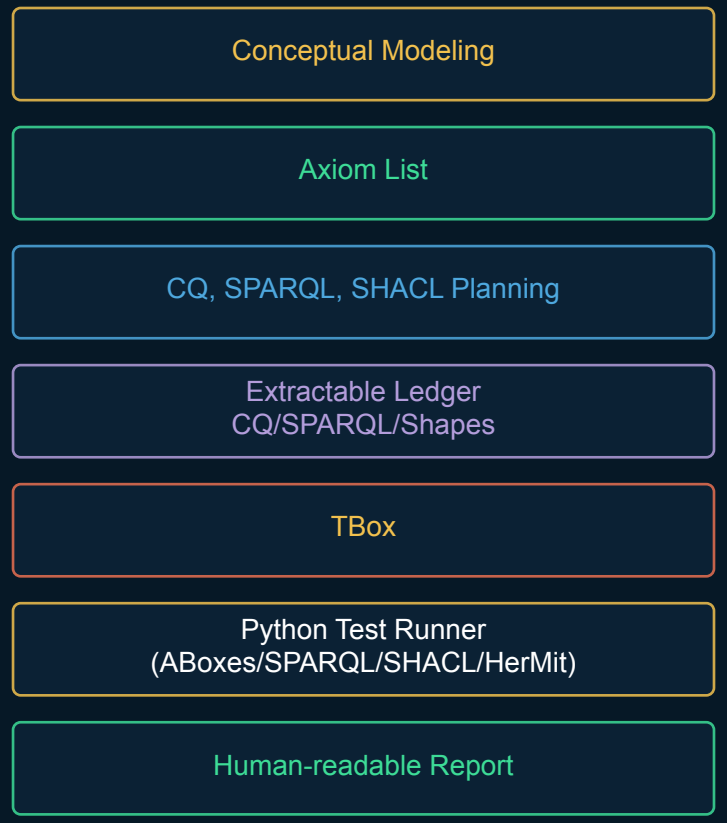
- RDF/TTL parsing proves only serialization health
- A reasoner can return consistent while real world requirements are missing
- A SPARQL query can return rows that do not prove the intended question
- A SHACL report can be technically correct but conceptually mis-scoped.

What is an Ontology?

Conceptualization.

Two answers.

- The most radical ontological question: why is there something rather than nothing?
- Before validating something
- What do you think you are validating?

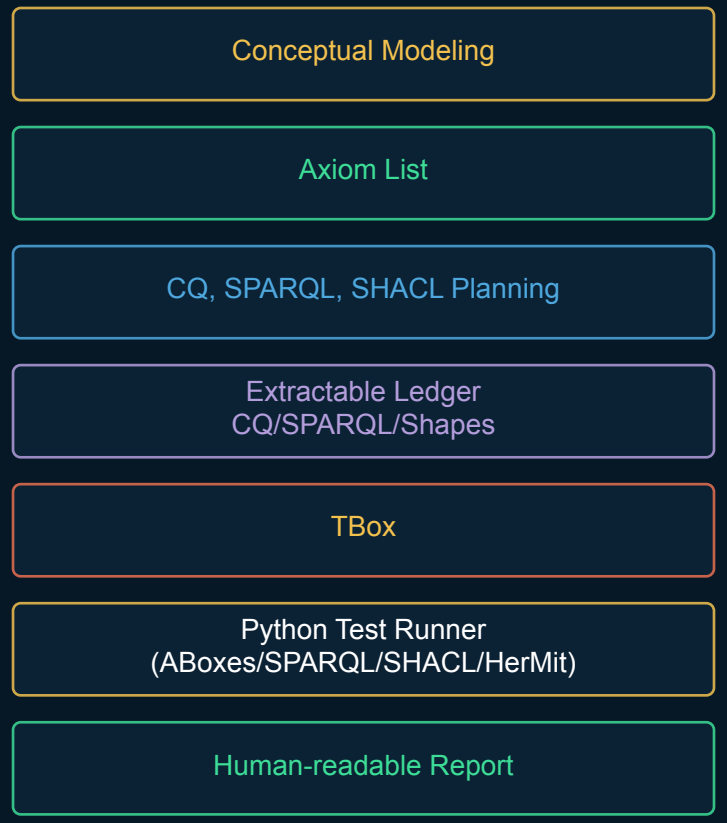


Ontology x OWL representation

Expression power x computational capability.

- Realist approach
- Entities, categories, relations, and structures that exist in reality
- Disciplined representation of those entities and relations
- One serialization or one OWL file
- An OWL selects part of the ontological theory
- Lossy formalization
- Constrained projection x richer ontology theory

Abstraction difference.

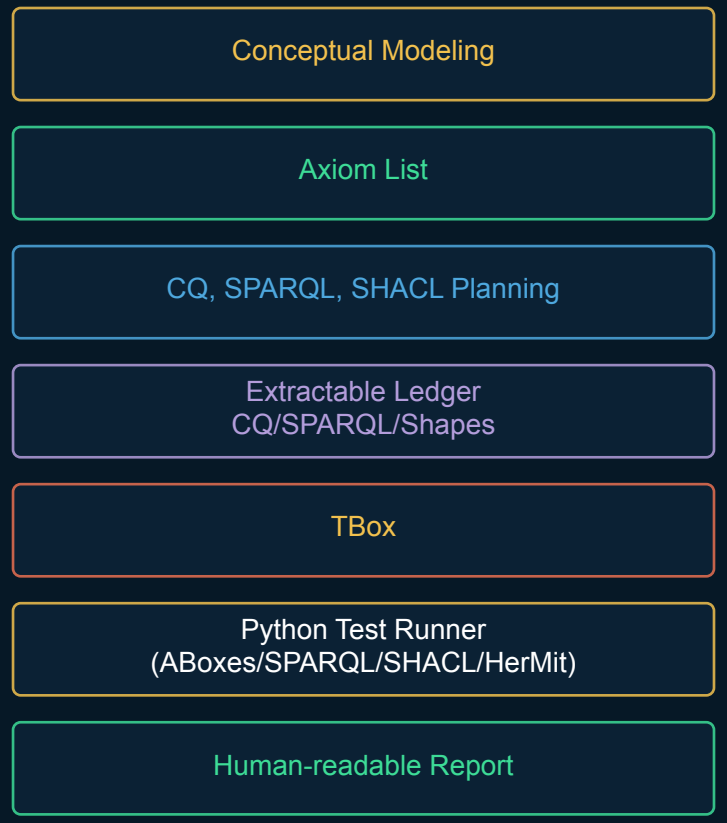


What Is the Strategy?

Aggregation becomes method.

Not the individual technologies — the disciplined aggregation of tools into a single evidence pipeline.

- BFO gives generic category discipline
- The Conceptual Modeling is grounded by BFO (7Buckets) and references
  - Foundations, entities, relationships and restrictions
- The Axiom List makes commitments explicit through
  - **class axioms** and **object-property and data-property axioms,** the actual categories used by OWL 2
- The Planning document allows to establish how CQ, SPARQL queries, Hermit safety, OWL restrictions, and SHACL shapes will be implemented
  - What / How will we test
- The Ledger makes tests extractable and scalable
  - CQ, SPARL, shapes, restrictions and Hermit safe test blocks
- Disciplined and realistic TBox
- Python runners turn documents into reproducible evidence



Phase 1 — The Conceptual Modeling

Before formalization, the domain must be made intellectually accountable.

Study = runway.

- Defines scope, scenarios, and intended use
- Collects authoritative references and existing ontologies
- Identifies entities, processes, qualities, roles, records, sites, and time
- Records uncertainties before they become hidden axioms.

Conceptual Modeling

Advantage of the Study

It makes later validation fair: tests evaluate declared commitments, not guessed intentions.

- Creates a readable source of truth
- Prevents arbitrary modeling decisions
- Shows why each important entity deserves representation
- Keeps references connected to requirements.

```
0. NAMESPACE DECLARATION

observation: <http://ncor.organization.ai/BFO-Process-Ledger/2026/ontologies/IC_ObservationProcessOntology#>
collection:  <http://ncor.organization.ai/BFO-Process-Ledger/2026/ontologies/IC_CollectionProcessOntology#>
analysis:    <http://ncor.organization.ai/BFO-Process-Ledger/2026/ontologies/IC_AnalysisProcessOntology#>
ta:          <http://ncor.organization.ai/BFO-Process-Ledger/2026/ontologies/
IC_ThreatAssessmentProcessOntology#>
dis:         <http://ncor.organization.ai/BFO-Process-Ledger/2026/ontologies/IC_DisseminationProcessOntology#>
sp7:         <http://ncor.organization.ai/BFO-Process-Ledger/2026/ontologies/top-level-structure-7Bucket#>
ncor:        <http://ncor.organization.ai/BFO-Process-Ledger/2026/ontologies/NCOR_CommonGraphVocabulary#>
prov:        <http://www.w3.org/ns/prov#>
```

2. POSITION IN THE INTELLIGENCE CYCLE

Observation → Collection → Analysis → Threat Assessment → Dissemination

Global semantic rule (phase separation), shared verbatim across the five-phase family:

Observation: perceives and records phenomena (no aggregation, no meaning).
Collection: aggregates / normalizes (no interpretation).
Analysis: interprets (no threat posture, severity, likelihood, priority).
Threat Assessment: evaluates risk posture (no discovery / interpretation).
Dissemination: conveys assessed products (no assessment).

Observation is the epistemic intake layer. All other phases operate on its informational residue, never on reality itself.

3. BFO AND 7BUCKET ASSUMPTIONS

This Study assumes top-level-structure-7Bucket-V4 as imported law, not as a document to be re-argued. In particular:

- The seven root buckets (sp7:GenericallyDependentContinuant, sp7:MaterialEntity, sp7:Process, sp7:Qualities [equivalentClass sp7:Quality], sp7:RealizableEntities, sp7:Site, sp7:TemporalRegion) are pairwise disjoint via owl:AllDisjointClasses. This is verified in the imported file itself, not merely doctrinal here.
- sp7:Role, sp7:Disposition, and sp7:Function are the three pairwise-disjoint subclasses of sp7:RealizableEntities. ObservationalSensitivityDisposition (Section 14) is explicitly typed as sp7:Disposition, never as an unqualified RealizableEntity, a Role, or a Function.
- Every native Observation class must be asserted under exactly one of these roots (directly or through Observation-native intermediate classes), with no cross-bucket double inheritance.

Naming convention (shared, invariant across the five-phase family):

Instance	– bounded run of a phase, classified as sp7:Process.
Interval	– temporal bound of the run, classified as sp7:TemporalRegion.
Context	– functional/governance setting, classified as sp7:Site.
Cell	– socio-technical executor as a single participant, classified as sp7:MaterialEntity.
QualityMarker	– run-level quality participant, classified as sp7:Quality.
Disposition	– realized capability inhering in a Cell, classified as sp7:Disposition.
GDC	– GenericallyDependentContinuant, for copyable informational content.

HermiT guardrail (shared 7-Bucket participant discipline, identical in form to J2BoundedProcess):

sp7:Process sp7:has_Participant exactly 2 .

Therefore, each ObservationInstance has exactly:

1. one ObservationCell, classified as sp7:MaterialEntity;
2. one ObservationQualityMarker, classified as sp7:Quality.

No other entity is a direct participant of ObservationInstance. No microprocess (Section 9) may carry its own sp7:has_Participant exact-count restriction — this is the exact mereological discipline whose violation was found and corrected in J2 Study V11 (Section 44 of that Study), and it is stated here up front rather than discovered later.

Phase 2 — The Axiom List

The Axiom List is the semantic contract between the Study and OWL implementation.

Commitment ledger.

- Every important statement receives an identifier
- Natural language remains visible
- Formal intention is recorded before serialization
- Validation destination is assigned early.

Advantage of the Axiom List

Supports traceability from domain claim to implementation.
Exposes HermiT-safety risks before they break reasoning.
Separates ontological axioms from validation policies.
Creates stable identifiers for downstream tests.

SABiO / SABiOx Inspiration

1. CORE PROCESS AXIOM

OBS-001 CORE observation:ObservationInstance subClassOf sp7:Process.
Meaning: roots the top-level Observation run at the shared 7Bucket Process root, not raw BFO — corrects V1's "ObservationInstanceProcess (BFO:Process)" framing.

OBS-002 CORE observation:ObservationInstance sp7:has_Participant exactly 1 observation:ObservationCell.
OBS-003 CORE observation:ObservationInstance sp7:has_Participant exactly 1 observation:ObservationQualityMarker.
OBS-004 SHACL-ONLY observation:ObservationInstance must have no direct sp7:has_Participant values other than the same-run observation:ObservationCell and observation:ObservationQualityMarker.
Meaning (OBS-002 through OBS-004): the exact two-participant guardrail, in the same form as J2BoundedProcess (J2-787, J2-788, J2-789).

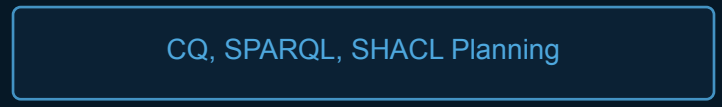
OBS-005 CORE observation:ObservationInstance sp7:occupies_TemporalRegion exactly 1 observation:ObservationInterval.
OBS-006 CORE observation:ObservationInstance sp7:occurs_In exactly 1 observation:ObservationContext.
OBS-007 CORE observation:ObservationInstance sp7:realizes some observation:ObservationalSensitivityDisposition.

Phase 3 — CQ–SPARQL–SHACL–HermiT Planning

Choose the proof.

Before writing tests, decide which kind of evidence each requirement needs.

OWL/HermiT for logical identity and satisfiability.
SPARQL for graph capabilities and anti-pattern discovery.
SHACL for closed-world conformance.
Human review for realism and source interpretation.



Advantage of Planning

It prevents one validation language from being misused as all validation languages.

Right tool, right claim.

Avoids over-strong OWL restrictions.
Avoids SHACL false positives from broad targets.
Avoids weak SPARQL queries with ambiguous evidence.
Avoids reports that mix blocked, failed, and not-applicable cases.

Competency Questions

Questions become tests.

- CQs express what the ontology must answer
- Positive CQs prove intended capability
- Negative CQs expose anti-patterns
- Cross-chain CQs test interoperability.

Advantage of CQs

They translate ontology quality into observable behavior.

Behavior, not slogans.

They test whether the ontology can do useful work.
They reveal missing relations and wrong property directions.
They help design ABoxes with positive and negative cases.
They support regression after patches.

3. COVERAGE LEVELS

LEVEL 1 — IDENTIFIER COVERAGE
Every current source identifier occurs in at least one fully expanded Ledger source_ids array.

LEVEL 2 — SEMANTIC BINDING
Each check validates the meaning of its source commitment, not merely a term occurring in the same paragraph.

LEVEL 3 — MECHANISM FITNESS
Each commitment is assigned to a mechanism appropriate to its open-world, closed-world, temporal, structural, provenance, or human-governance semantics.

LEVEL 4 — CONVERSION SPECIFICATION
Each check uses an approved SPARQL, SHACL, OWL/HermiT, implementation, or review template.

LEVEL 5 — EXECUTABLE SERIALIZATION
All placeholders are replaced by exact ontology IRIs; query and shape files parse; import resolution and manifest activation succeed.

LEVEL 6 — EXECUTION PROOF
The checks execute against TBox, VALID ABox, MIXED ABox, CROSS-CHAIN ABox, and bridge graphs with expected evidence.

5.6 HUMAN REVIEW

Human review remains primary for substantive legality, ethics, classification correctness, material uncertainty, analytic adequacy, and whether an asserted trigger accurately represents the real-world condition. Machine validation checks the existence and traceability of the review record.

6. PROFILE ACTIVATION POLICY

CORE checks are always loaded.
STRICT checks are loaded for approved or command-grade validation datasets.
COMPLETED-RUN checks activate only for runs explicitly declared completed.
CONDITIONAL and STRICT-WHEN-* checks require an external manifest activation or an approved domain trigger.
CROSS-CHAIN checks require the graph set declared in the manifest.
BRIDGE-PREPARATION checks are executed on the bridge artifact, not by adding PROV-O typing to the native J2 ontology.

The runner must never infer NOT_APPLICABLE merely from absence of the required output. Absence under an active profile is a violation. Absence under an inactive profile is NOT_APPLICABLE.

The validation manifest is runner metadata and does not alter the BFO category of any domain entity.

```
8. PARAMETERIZED SPARQL CONVERSION LIBRARY

[BEGIN SPARQL TEMPLATE Q-TBOX-ASK]
ASK {
  {{SUBJECT_CLASS}} rdfs:subClassOf+ {{OBJECT_CLASS}} .
}
[END SPARQL TEMPLATE Q-TBOX-ASK]

[BEGIN SPARQL TEMPLATE Q-TBOX-NOT-ASK]
ASK {
  FILTER NOT EXISTS {
    {{SUBJECT_CLASS}} rdfs:subClassOf+ {{FORBIDDEN_CLASS}} .
  }
}
[END SPARQL TEMPLATE Q-TBOX-NOT-ASK]
```

```
9. PARAMETERIZED SHACL CONVERSION LIBRARY

[BEGIN SHACL TEMPLATE SH-REQUIRED-PROPERTY]
{{SHAPE_IRI}} a sh:NodeShape ;
  sh:targetClass {{FOCUS_CLASS}} ;
  sh:property [
    sh:path {{PROPERTY}} ;
    sh:minCount 1 ;
    sh:class {{VALUE_CLASS}}
  ] .
[END SHACL TEMPLATE SH-REQUIRED-PROPERTY]

[BEGIN SHACL TEMPLATE SH-EXACT-COUNT]
{{SHAPE_IRI}} a sh:NodeShape ;
  sh:targetClass {{FOCUS_CLASS}} ;
  sh:property [
    sh:path {{PROPERTY}} ;
    sh:qualifiedValueShape [ sh:class {{VALUE_CLASS}} ] ;
    sh:qualifiedMinCount {{COUNT}} ;
    sh:qualifiedMaxCount {{COUNT}}
  ] .
```

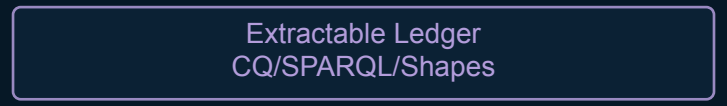
Phase 4 — The Extractable Ledger

The Ledger is a human-readable document that can also drive software execution.

Readable + executable.

Stores CQ text, rationale, query code, shape code, and expected result.
Provides markers for automated extraction.
Keeps pass criteria near the test.
Feeds the human-readable report.

Advantage of the Ledger



It protects validation from becoming undocumented code.

Supports reproducibility.
Supports audit and teaching.
Supports automated report generation.
Supports forensic debugging when a test fails.

Tests with memory.

Ledger Anatomy

Each record must support both machines and humans.

Every test has a why.

ID and requirement.
Why it exists.
What it validates.
Executable SPARQL or SHACL block.
Expected result and pass criterion.
Profile and proof type.

```
SPARQL_RECORD_BEGIN
CQ_ID = CQ-J2-130
SPARQL_ID = SPARQL-ABOX-J2-032
RECORD_TYPE = SPARQL_QUERY
EXECUTABLE = true
EXECUTION_HANDLER = sparql
CHECK_FAMILY = CARRIAGE_PARTHOOD_LEAKAGE
PRIMARY_MECHANISM = SPARQL_SELECT
TARGET_GRAPHS = ABOX_VALID,ABOX_MIXED
PROFILE = ANTI-PATTERN
ACTIVATION_RULE = ALWAYS
EXPECTED_RESULT = empty SELECT
SEMANTIC_BINDING = This record validates its declared CQ(s) by checking the exact class, property, path, or implementation gate named by the CQ;
weaker substitutes are not acceptable.
QUERY_TEXT_BEGIN
SELECT DISTINCT ?carrier ?content WHERE { ?carrier sp7:is_CarrierOf ?content ; j2:hasMaterialPart ?content . ?content rdf:type/rdfs:subClassOf*
sp7:GenericallyDependentContinuant . } ORDER BY ?carrier ?content
QUERY_TEXT_END
STATUS_VALUES_ALLOWED =
PASS,FAIL,NOT_APPLICABLE,TEMPORAL_MODEL_REQUIRED,IMPORT_IRI_UNRESOLVED,GRAPH_SET_INCOMPLETE,MATERIALIZATION_REQUIRED,HUMAN_REVIEW_REQUIRED,EXECUTAB-
LE_BLOCK_MISSING,EXECUTABLE_BLOCK_PARSE_ERROR,SOURCE_BINDING_ERROR,VOCABULARY_ERROR,NOT_RUN
SPARQL_RECORD_END
```

Phase 5 — TBox Implementation (rdf/ttl)

Implementation is not the beginning; it is the serialized result of earlier commitments.

Files are outputs.

Classes and properties must trace back to terms and axioms.
Comments should explain meaning, BFO type, and rationale.
Imports, namespaces, and version IRIs must be controlled.
Generated files are parsed, reloaded, and tested before release.

IC_ObservationProcessOntology (http://ncor.organization.ai/BFO-Process-Ledger/2026/ontologies/IC_ObservationProcessOntology) : [/...

File Edit View Reasoner Tools Refactor Window Ontop Help

IC_ObservationProcessOntology (http://ncor.organization.ai/BFO-Process-Ledger/2026/ontologies/IC_ObservationProcessOntology)

Active ontology × Entities × Individuals by class × DL Query × Individual Hierarchy Tab ×

Ontology header:

Ontology IRI http://ncor.organization.ai/BFO-Process-Ledger/2026/ontologies/IC_ObservationProcessOntology

Ontology Version IRI e.g. http://ncor.organization.ai/BFO-Process-Ledger/2026/ontologies/IC_ObservationProcessOntology/1.0.0

Annotations

rdfs:comment

NCOR — Enterprise Intelligence Cycle (BFO 7-Bucket disciplined)
Ontology: Observation Process Ontology

This ontology was constructd from the Observation Process study. It imports the 7-Bucket root ontology and introduces ONLY child classes under those BFO root buckets.

Ontology metrics:

Metrics

Axiom	253
Logical axiom count	112
Declaration axioms count	65
Class count	40
Object property count	27
Data property count	0
Individual count	0
Annotation Property count	2

Class axioms

SubClassOf	68
EquivalentClasses	0
DisjointClasses	3

Ontology imports | Ontology Prefixes | General class axioms

Imported ontologies:

Direct Imports

<http://ncor.organization.ai/BFO-Process-Ledger/2026/ontologies/top-level-structure-7Buckets>

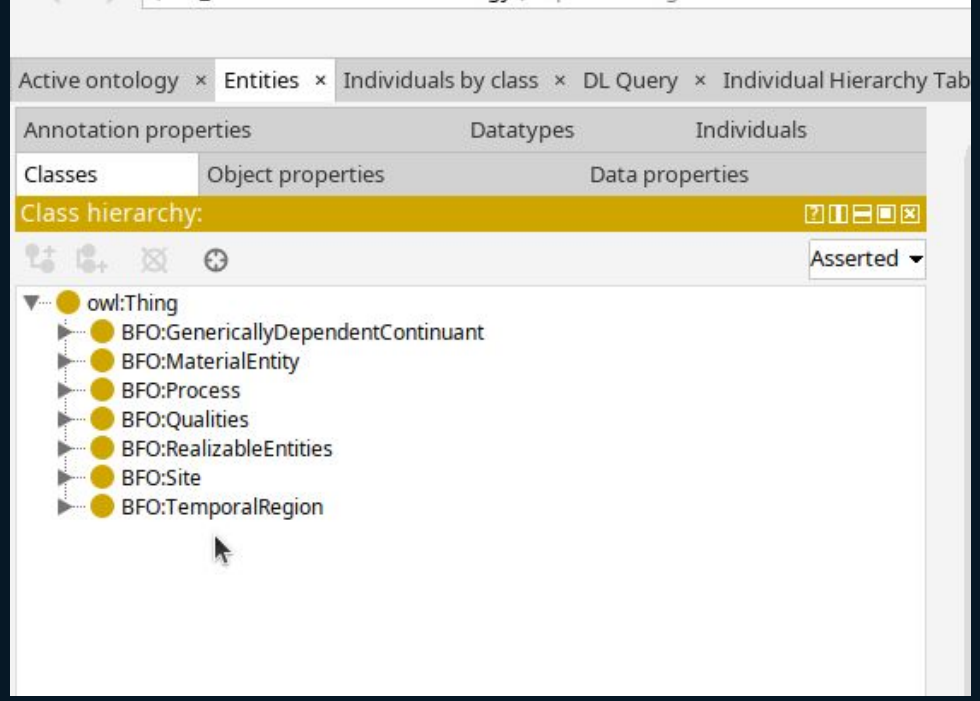
top-level-structure-7Buckets (66 axioms, 27 logical axioms)

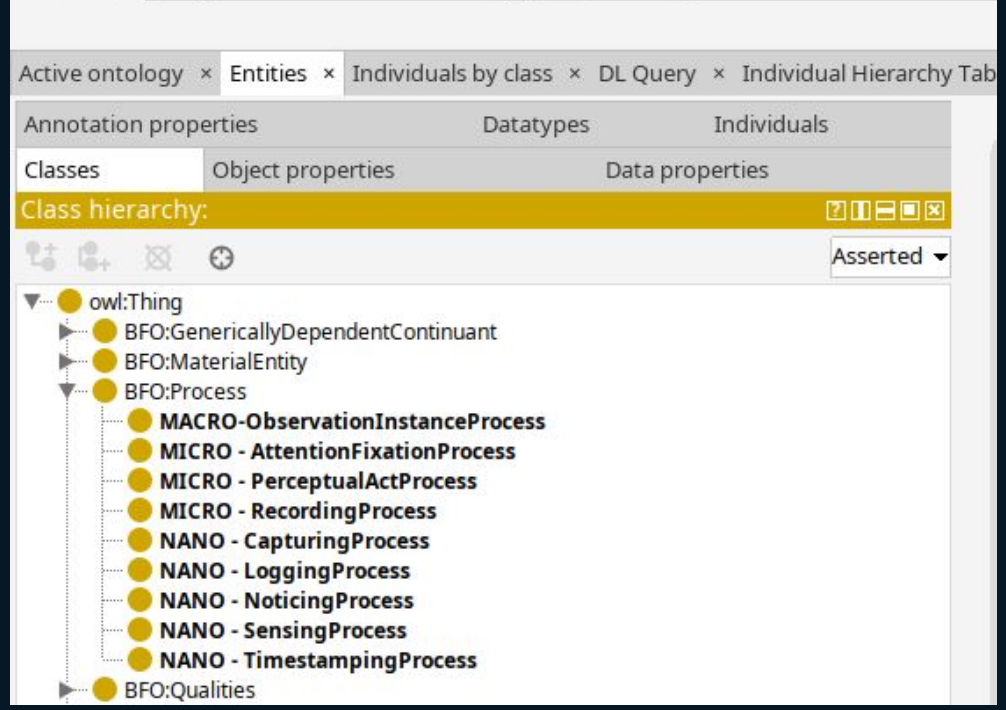
Ontology IRI: <http://ncor.organization.ai/BFO-Process-Ledger/2026/ontologies/top-level-structure-7Buckets>

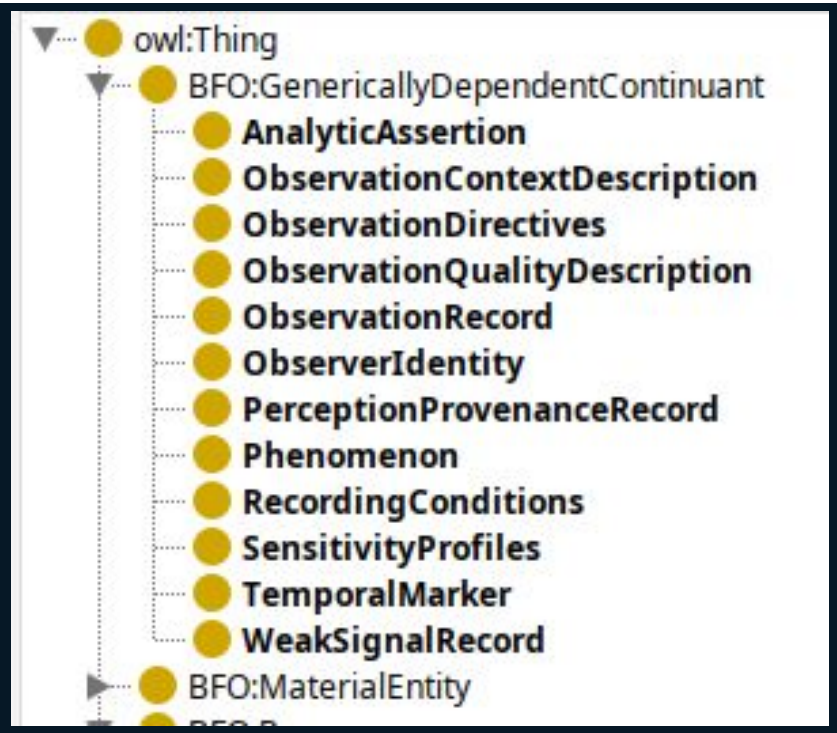
Location: /home/rodriquez/Desktop/2026/NCOR/Intelligence/Intel_IntelligenceCycle/IC_ObservationProcessOntology/Before_July_23th/Original/DeliveryVersion/top-level-structure-7Buckets.ttl

Indirect Imports

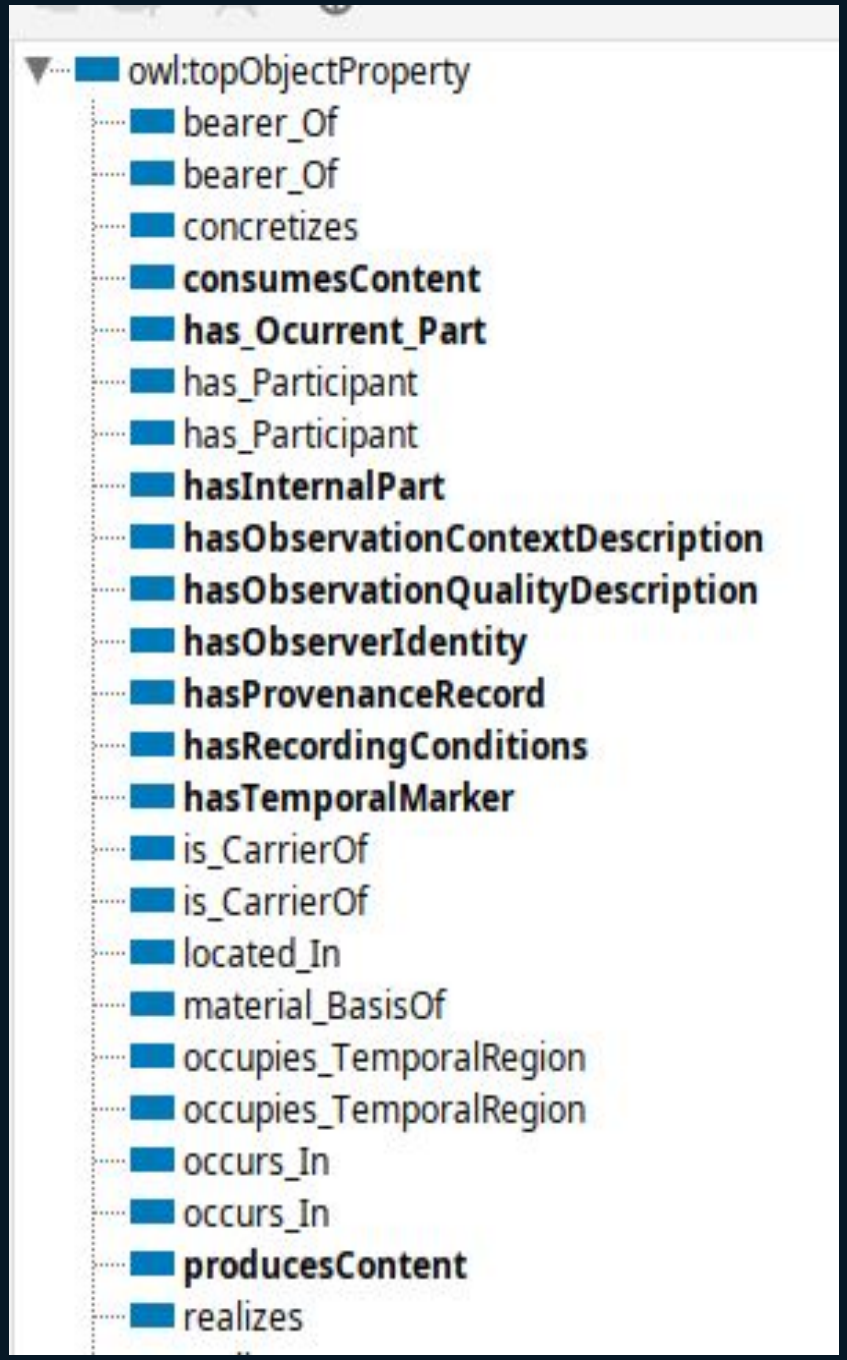
Reasoner active Show Inferences


Active ontology
Entities
Individuals by class
DL Query
Individual Hierarchy Tab
Annotation properties
Datatypes
Individuals
Classes
Object properties
Data properties
Class hierarchy:
Asserted
owl:Thing
BFO:GenericallyDependentContinuant
BFO:MaterialEntity
BFO:Process
BFO:Qualities
BFO:RealizableEntities
BFO:Site
BFO:TemporalRegion


Active ontology
Entities
Individuals by class
DL Query
Individual Hierarchy Tab
Annotation properties
Datatypes
Individuals
Classes
Object properties
Data properties
Class hierarchy:
Asserted
owl:Thing
BFO:GenericallyDependentContinuant
BFO:MaterialEntity
BFO:Process
MACRO-ObservationInstanceProcess
MICRO - AttentionFixationProcess
MICRO - PerceptualActProcess
MICRO - RecordingProcess
NANO - CapturingProcess
NANO - LoggingProcess
NANO - NoticingProcess
NANO - SensingProcess
NANO - TimestampingProcess
BFO:Qualities


owl:Thing
BFO:GenericallyDependentContinuant
AnalyticAssertion
ObservationContextDescription
ObservationDirectives
ObservationQualityDescription
ObservationRecord
ObserverIdentity
PerceptionProvenanceRecord
Phenomenon
RecordingConditions
SensitivityProfiles
TemporalMarker
WeakSignalRecord
BFO:MaterialEntity


WeakSignalRecord
BFO:MaterialEntity
BFO:Process
BFO:Qualities
ObservationQualityMarker
BFO:RealizableEntities
ObservationalSensitivityDisposition
BFO:Site
ObservationContext
BFO:TemporalRegion
ObservationInterval


owl:topObjectProperty
bearer_Of
bearer_Of
concretizes
consumesContent
has_Ocurrent_Part
has_Participant
has_Participant
hasInternalPart
hasObservationContextDescription
hasObservationQualityDescription
hasObserverIdentity
hasProvenanceRecord
hasRecordingConditions
hasTemporalMarker
is_CarrierOf
is_CarrierOf
located_In
material_BasisOf
occupies_TemporalRegion
occupies_TemporalRegion
occurs_In
occurs_In
producesContent
realizes

Class hierarchy: MACRO-ObservationInsta

Asserted

- owl:Thing
  - BFO:GenericallyDependentContinuant
    - **AnalyticAssertion**
    - **ObservationContextDescription**
    - **ObservationDirectives**
    - **ObservationQualityDescription**
    - **ObservationRecord**
    - **ObserverIdentity**
    - **PerceptionProvenanceRecord**
    - **Phenomenon**
    - **RecordingConditions**
    - **SensitivityProfiles**
    - **TemporalMarker**
    - **WeakSignalRecord**
  - BFO:MaterialEntity
  - BFO:Process
    - **MACRO-ObservationInstanceProcess**
    - **MICRO - AttentionFixationProcess**
    - **MICRO - PerceptualActProcess**
    - **MICRO - RecordingProcess**
    - **NANO - CapturingProcess**
    - **NANO - LoggingProcess**
    - **NANO - NoticingProcess**
    - **NANO - SensingProcess**
    - **NANO - TimestampingProcess**
  - BFO:Qualities

Annotations: MACRO-ObservationInstanceProcess

Annotations

rdfs:label

MACRO-ObservationInstanceProcess

rdfs:comment

Name: ObservationInstance.
Meaning in the intelligence cycle: one bounded run of disciplined perception and recording (no aggregation, no interpretation), producing atomic records + descriptive provenance for Collection.
BFO type: Process (sp7:Process).
Why classified as this type: it is a time-extended occurrent that occupies a temporal region, occurs in a site, has exactly two participants (Cell + QualityMarker), realizes a disposition, and produces GDC artifacts.

Description: MACRO-ObservationInstanceProcess

Equivalent To

SubClass Of

- **BFO:Process**
- **has_Ocurrent_Part some 'MICRO - AttentionFixationProcess'**
- **has_Ocurrent_Part some 'MICRO - PerceptualActProcess'**
- **has_Ocurrent_Part some 'MICRO - RecordingProcess'**
- **has_Participant exactly 1 ObservationCell**
- **has_Participant exactly 1 ObservationQualityMarker**

Phase 6 - Building a Python Runner

ABoxes are experiments.

ABox Profiles

CLEAN/VALID data must pass.
VIOLATIONS data must fail in named ways.
MIXED data tests localization of defects.
CROSS-CHAIN data proves interoperability.
TBOX-ONLY isolates logical health.

Advantage of ABox Profiles

The same ontology can be tested under different scientific conditions.

Profile = question.

They clarify what a result means.
They prevent false failure classification.
They make negative testing legitimate.
They support scale testing without changing the TBox.

Phase 6 - Building a Python Runner

Extracting SPARQL and Shapes from the Ledger

Blocks are marked on the Ledger Document
rdflib, owlready2, and pyshacl
Confront TBox, ABox
Do not need a ttl file, but the Ledger is extractable

Phase 6 - Building a Python Runner

Generate Reports

Statistics
Coverage of TBox, and ABoxes
ID, Why here, What check, Type, Expected result, Real result, criteria to pass, classification, proof
Human review

Phase 6 - Building a Python Runner

Destroying virtual ABoxes, persisting hash code, delivery products

Txt, JSON
Summary reports
Full reports
Triples as proofs
pyshacl classification

Expected Contributions

1. **Requirements-to-Evidence Traceability**
2. **Validation Technologies Matched to Logical Purpose**
3. **Executable Competency Questions and Constraints**
4. **Reproducible Validation with Controlled ABoxes**
5. **Backward Diagnosis of Semantic Defects**

Future Work

1. Automated Requirement-to-Test Generation
2. Benchmarking and Standardization of Ontology Validation

Thank You!

Luís Antonio de Almeida Rodriguez - PhD

rodriguezlaar@gmail.com